A Beginner's Guide to Loop Unrolling in LLVM

Justin Fargnoli

NVIDIA

2026 LLVM Developers' Meeting

Why Talk About Loop Unrolling?

- The unrolling decisions we make can have a drastic impact on key compiler metrics:
  - Runtime: loops often contain a program's hot paths, so transforming them can have a large impact on performance.
  - Compile time: excessive unrolling can dramatically increase the amount of IR that later passes must process.
  - Code size: an overly aggressive unroll factor can multiply the size of an already large loop body.
- Unrolling can enable optimizations that were not possible on the original loop and motivate optimizations tailored to unrolled code.

Who Is This Talk For?

- You know what loop unrolling is.
- You're familiar with LLVM's modular design: optimization passes focus on specific transformations, and targets can tune their behavior through TargetTransformInfo (TTI).
- But you aren't an expert on how each kind of unrolling rewrites the IR, how LLVM decides to unroll, or how unrolling interacts with other passes.

Table of Contents

Background

Unrolling Different Kinds of Loops

Deciding to Unroll

Pass Interactions at -O3

Notes for the Road

Table of Contents

Background
- Compilation Flow
- Structure of a Loop
- Terminology

Unrolling Different Kinds of Loops

Deciding to Unroll

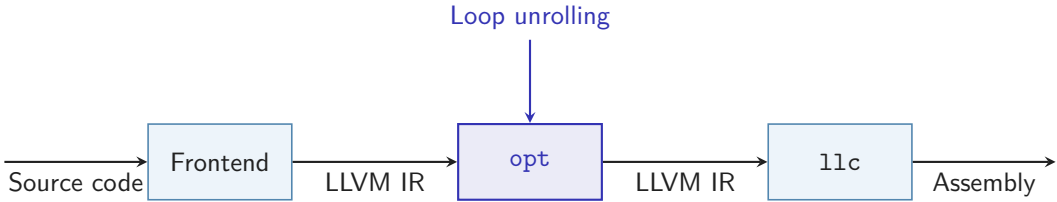
Pass Interactions at -O3

Notes for the Road

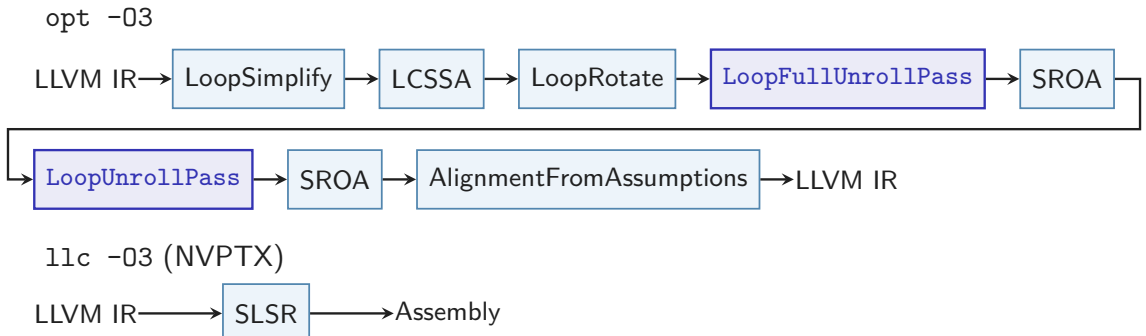
Compilation Flow



Compilation Flow



LLVM Pass Pipelines



Other passes are omitted for brevity; shown passes retain their order.

Table of Contents

Background
- Compilation Flow
- Structure of a Loop
- Terminology

Unrolling Different Kinds of Loops

Deciding to Unroll

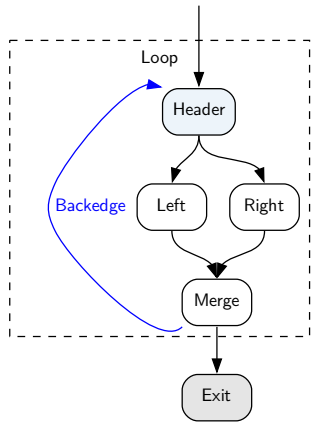
Pass Interactions at -O3

Notes for the Road

What Is a Loop in LLVM?

`LoopInfo` identifies **natural loops**: cyclic sets of basic blocks with:

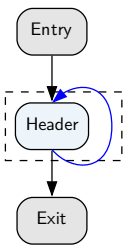
- **Strongly connected.** In the subgraph, every block can reach every other.
- **One header.** Every edge from outside enters at the header.
- **Maximal for that header.** No larger set satisfies both properties while keeping the same header.



A Single-Block Loop

- A loop can contain just **one basic block**.
- That block is the **header**, and its **backedge** returns to itself.

Control flow



LLVM IR

```
entry:
  br label %header

header:
  %i = phi i32 [ 0, %entry ], [ %inc, %header ]
  %inc = add i32 %i, 1
  %more = icmp slt i32 %inc, %n
  br i1 %more, label %header, label %exit

exit:
  ret void
```

Table of Contents

Background
- Compilation Flow
- Structure of a Loop
- Terminology

Unrolling Different Kinds of Loops

Deciding to Unroll

Pass Interactions at -O3

Notes for the Road

Induction Variable

- An **induction variable (IV)** changes by a fixed step each iteration.
- In this example, **`i` is the IV**: it starts at 0 and increases by 1.

Source

```
for (int i = 0; i < 2; ++i)
  foo(i);
```

IR

```
body:
  %i = phi i32 [ 0, %entry ], [ %inc, %body ]
  call void @foo(i32 noundef %i)
  %inc = add nuw nsw i32 %i, 1
  %cmp = icmp slt i32 %inc, 2
  br i1 %cmp, label %body, label %exit

exit:
  ret void
```

Trip Count

- The **trip count** is the number of times the loop header executes.
- In this example, the trip count is **2**[*].

Source

```
for (int i = 0; i < 2; ++i)
  foo(i);
```

IR

```
body:
  %i = phi i32 [ 0, %entry ], [ %inc, %body ]
  call void @foo(i32 noundef %i)
  %inc = add nuw nsw i32 %i, 1
  %cmp = icmp slt i32 %inc, 2
  br i1 %cmp, label %body, label %exit

exit:
  ret void
```

[*]IR shown after `LoopRotate`. Before rotation, the source loop's header executes 3 times.

Different Kinds of Trip Counts

Static

The exact count is a compile-time constant.

```
for (int i = 0; i < 2; ++i)
  foo(i);
```

Count: 2

Upper bound

A maximum count is a compile-time constant.

```
n = min(n, 3);
for (int i = 0; i < n; ++i)
  foo(i);
```

Count: 0, 1, 2, or 3

Runtime

The exact count depends on a runtime value.

```
for (int i = 0; i < n; ++i)
  foo(i);
```

Count: $\max(n, 0)$

Counts shown after `LoopRotate`.

Scalar Evolution (SCEV)

ScalarEvolution is an LLVM analysis pass that analyzes how values change across loop iterations.

- **Recognizes induction variables** and models their start and step.
- **Derives trip counts** from recurrences and exit conditions when possible.

Constant trip count

```
for (int i = 0; i < 4; ++i)
  foo(i);
```

IV: $\{0, +, 1\}$

Trip count: 4[*].

Runtime trip count

```
for (int i = n; i > 0; i -= 3)
  foo(i);
```

IV: $\{n, +, -3\}$

Trip count[*]: `(1 + ((-1 + %n) /u 3))`

[*] after `LoopRotate`; the runtime expression assumes `n > 0`.

Unroll Factor

- The **unroll factor** (aka **unroll count**) is the number of original iterations grouped into an unrolled body.
- In the example, there are **two original iterations per group** and **two groups**.

Input

```
for (int i = 0; i < 4; ++i)
  foo(i);
```

Unrolled by a factor of 2

```
for (int i = 0; i < 4; i += 2) {
  foo(i);
  foo(i + 1);
}
```

Table of Contents

Background

Unrolling Different Kinds of Loops

Deciding to Unroll

Pass Interactions at -O3

Notes for the Road

Preprocessing

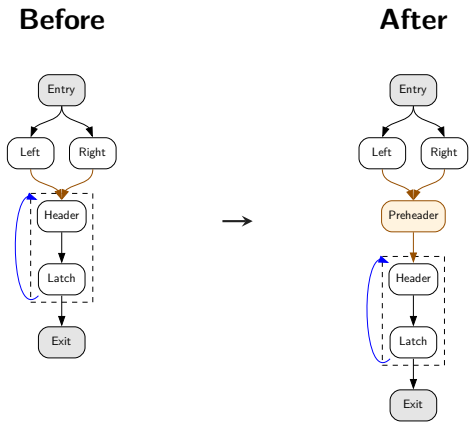
`opt -O3`



Other passes are omitted for brevity; shown passes retain their order.

Loop Simplify Form

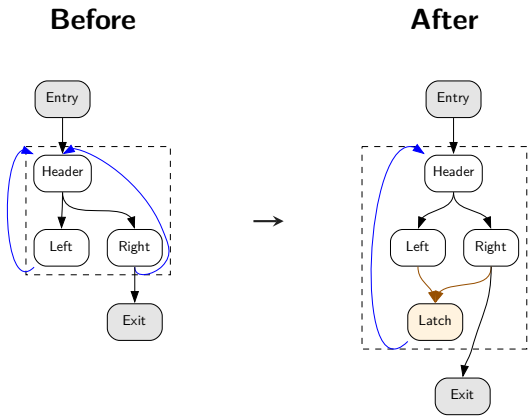
- A **preheader**: the header's only predecessor outside the loop; its only successor is the header.



The `LoopSimplify` pass performs this transformation.

Loop Simplify Form

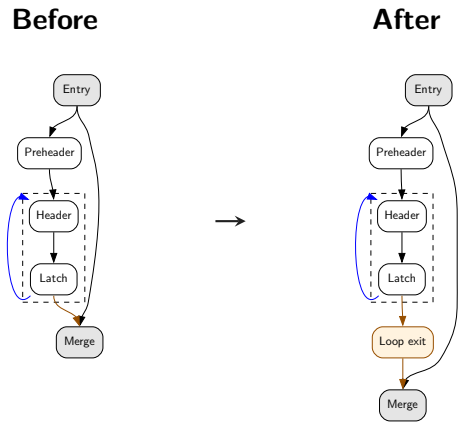
- A **preheader**: the header's only predecessor outside the loop; its only successor is the header.
- A **single backedge**, from one **latch**: a block inside the loop that branches back to the header.



The `LoopSimplify` pass performs this transformation.

Loop Simplify Form

- A **preheader**: the header's only predecessor outside the loop; its only successor is the header.
- A **single backedge**, from one **latch**: a block inside the loop that branches back to the header.
- **Dedicated exits**: all their predecessors are inside the loop.



The `LoopSimplify` pass performs this transformation.

Loop-Closed SSA (LCSSA)

- Loop values used outside the loop pass through **PHIs in exit blocks**.
- `LCSSA` inserts these PHIs; unrolling updates them when it reconnects loop results.

Source

```
int accumulator = 0;
for (int i = 0; i < 4; ++i)
  accumulator += i;
return accumulator;
```

Before LCSSA

```
loop:
  %i = phi i32 [ 0, %entry ], [ %inc, %loop ]
  %accumulator = phi i32 [ 0, %entry ], [ %next, %loop ]
  %next = add i32 %accumulator, %i
  %inc = add i32 %i, 1
  %more = icmp ult i32 %inc, 4
  br i1 %more, label %loop, label %exit

exit:
  ret i32 %next
```

Loop-Closed SSA (LCSSA)

- Loop values used outside the loop pass through **PHIs in exit blocks**.
- `LCSSA` inserts these PHIs; unrolling updates them when it reconnects loop results.

Source

```
int accumulator = 0;
for (int i = 0; i < 4; ++i)
  accumulator += i;
return accumulator;
```

After LCSSA

```
loop:
  %i = phi i32 [ 0, %entry ], [ %inc, %loop ]
  %accumulator = phi i32 [ 0, %entry ], [ %next, %loop ]
  %next = add i32 %accumulator, %i
  %inc = add i32 %i, 1
  %more = icmp ult i32 %inc, 4
  br i1 %more, label %loop, label %exit

exit:
  %next.lcssa = phi i32 [ %next, %loop ]
  ret i32 %next.lcssa
```

Preprocessing

`opt -O3`



Other passes are omitted for brevity; shown passes retain their order.

Unrolling a Loop

Source

```
for (int i = 0; i < 2; ++i)
  foo(i);
```

IR

```
body:
  %i = phi i32 [ 0, %entry ], [ %inc, %body ]
  call void @foo(i32 noundef %i)
  %inc = add nuw nsw i32 %i, 1
  %cmp = icmp slt i32 %inc, 2
  br i1 %cmp, label %body, label %exit

exit:
  ret void
```

Unrolling a Loop

- How can we remove the loop structure?

Source

```
for (int i = 0; i < 2; ++i)
  foo(i);
```

IR

```
body:
  %i = phi i32 [ 0, %entry ], [ %inc, %body ]
  call void @foo(i32 noundef %i)
  %inc = add nuw nsw i32 %i, 1
  %cmp = icmp slt i32 %inc, 2
  br i1 %cmp, label %body, label %exit

exit:
  ret void
```

Fixing up the Copied Loop Bodies

Before unrolling

```
body:
  %i = phi i32 [ 0, %entry ], [ %inc, %body ]
  call void @foo(i32 noundef %i)
  %inc = add nuw nsw i32 %i, 1
  %cmp = icmp slt i32 %inc, 2
  br i1 %cmp, label %body, label %exit

exit:
  ret void
```

Copied bodies (incomplete)

```
body:
  %i = phi i32 [ 0, %entry ], [ %inc, %body ]
  call void @foo(i32 noundef %i)
  %inc = add nuw nsw i32 %i, 1
  %cmp = icmp slt i32 %inc, 2
  br i1 %cmp, label %body, label %exit

body.1:
  %i.1 = phi i32 [ 0, %entry ], [ %inc, %body ]
  call void @foo(i32 noundef %i)
  %inc.1 = add nuw nsw i32 %i, 1
  %cmp.1 = icmp slt i32 %inc, 2
  br i1 %cmp, label %body, label %exit

exit:
  ret void
```

Fixing up the Copied Loop Bodies

- × The copied PHI and instructions still refer to the original loop.

Copied bodies (incomplete)

```
body:
  %i = phi i32 [ 0, %entry ], [ %inc, %body ]
  call void @foo(i32 noundef %i)
  %inc = add nuw nsw i32 %i, 1
  %cmp = icmp slt i32 %inc, 2
  br i1 %cmp, label %body, label %exit

body.1:
  %i.1 = phi i32 [ 0, %entry ], [ %inc, %body ]
  call void @foo(i32 noundef %i)
  %inc.1 = add nuw nsw i32 %i, 1
  %cmp.1 = icmp slt i32 %inc, 2
  br i1 %cmp, label %body, label %exit

exit:
  ret void
```

Fixing up the Copied Loop Bodies

- ✓ Replace the copied PHI with `%inc`, and remap the copied instructions.
- × The original PHI still expects a value from the backedge.

Copied bodies (incomplete)

```
body:
  %i = phi i32 [ 0, %entry ], [ %inc, %body ]
  call void @foo(i32 noundef %i)
  %inc = add nuw nsw i32 %i, 1
  %cmp = icmp slt i32 %inc, 2
  br i1 %cmp, label %body, label %exit

body.1:
  %i.1 = phi i32 [ 0, %entry ], [ %inc, %body ]
  call void @foo(i32 noundef %i)
  %inc.1 = add nuw nsw i32 %i, 1
  %cmp.1 = icmp slt i32 %inc, 2
  br i1 %cmp, label %body, label %exit

exit:
  ret void
```

Updated values (incomplete)

```
body:
  %i = phi i32 [ 0, %entry ], [ %inc, %body ]
  call void @foo(i32 noundef %i)
  %inc = add nuw nsw i32 %i, 1
  %cmp = icmp slt i32 %inc, 2
  br i1 %cmp, label %body, label %exit

body.1:

  call void @foo(i32 noundef %inc)
  %inc.1 = add nuw nsw i32 %inc, 1
  %cmp.1 = icmp slt i32 %inc.1, 2
  br i1 %cmp.1, label %body.1, label %exit

exit:
  ret void
```

Fixing up the Copied Loop Bodies

- ✓ Replace the copied PHI with `%inc`, and remap the copied instructions.
- ✓ Replace the original PHI with its initial value, `0`.

Updated values (incomplete)

```
body:
  %i = phi i32 [ 0, %entry ], [ %inc, %body ]
  call void @foo(i32 noundef %i)
  %inc = add nuw nsw i32 %i, 1
  %cmp = icmp slt i32 %inc, 2
  br i1 %cmp, label %body, label %exit

body.1:

  call void @foo(i32 noundef %inc)
  %inc.1 = add nuw nsw i32 %inc, 1
  %cmp.1 = icmp slt i32 %inc.1, 2
  br i1 %cmp.1, label %body.1, label %exit

exit:
  ret void
```

Resolved PHI (incomplete)

```
body:

  call void @foo(i32 noundef 0)
  %inc = add nuw nsw i32 0, 1
  %cmp = icmp slt i32 %inc, 2
  br i1 %cmp, label %body, label %exit

body.1:

  call void @foo(i32 noundef %inc)
  %inc.1 = add nuw nsw i32 %inc, 1
  %cmp.1 = icmp slt i32 %inc.1, 2
  br i1 %cmp.1, label %body.1, label %exit

exit:
  ret void
```

Fixing up the Copied Loop Bodies

- ✓ Replace the copied PHI with `%inc`, and remap the copied instructions.
- ✓ Replace the original PHI with its initial value, `0`.
- × The branches still loop back, although we know there are exactly two iterations.

Resolved PHI (incomplete)

```
body:

  call void @foo(i32 noundef 0)
  %inc = add nuw nsw i32 0, 1
  %cmp = icmp slt i32 %inc, 2
  br i1 %cmp, label %body, label %exit

body.1:

  call void @foo(i32 noundef %inc)
  %inc.1 = add nuw nsw i32 %inc, 1
  %cmp.1 = icmp slt i32 %inc.1, 2
  br i1 %cmp.1, label %body.1, label %exit

exit:
  ret void
```

Fixing up the Copied Loop Bodies

- ✓ Replace the copied PHI with `%inc`, and remap the copied instructions.
- ✓ Replace the original PHI with its initial value, `0`.
- ✓ Use the known trip count to fold the branches: continue after the first body; exit after the second.

Resolved PHI (incomplete)

```
body:

  call void @foo(i32 noundef 0)
  %inc = add nuw nsw i32 0, 1
  %cmp = icmp slt i32 %inc, 2
  br i1 %cmp, label %body, label %exit

body.1:

  call void @foo(i32 noundef %inc)
  %inc.1 = add nuw nsw i32 %inc, 1
  %cmp.1 = icmp slt i32 %inc.1, 2
  br i1 %cmp.1, label %body.1, label %exit

exit:
  ret void
```

Unrolled IR

```
body:

  call void @foo(i32 noundef 0)
  %inc = add nuw nsw i32 0, 1
  %cmp = icmp slt i32 %inc, 2
  br label %body.1

body.1:

  call void @foo(i32 noundef %inc)
  %inc.1 = add nuw nsw i32 %inc, 1
  %cmp.1 = icmp slt i32 %inc.1, 2
  br label %exit

exit:
  ret void
```

Unrolling the Loop

- Is that all we need to do to unroll this loop?
  - Yes—the backedge is gone, but we can do more.

Before unrolling

```
body:
  %i = phi i32 [ 0, %entry ], [ %inc, %body ]
  call void @foo(i32 noundef %i)
  %inc = add nuw nsw i32 %i, 1
  %cmp = icmp slt i32 %inc, 2
  br i1 %cmp, label %body, label %exit

exit:
  ret void
```

Unrolled IR

```
body:

  call void @foo(i32 noundef 0)
  %inc = add nuw nsw i32 0, 1
  %cmp = icmp slt i32 %inc, 2
  br label %body.1

body.1:

  call void @foo(i32 noundef %inc)
  %inc.1 = add nuw nsw i32 %inc, 1
  %cmp.1 = icmp slt i32 %inc.1, 2
  br label %exit

exit:
  ret void
```

Cleaning up the Unrolled Code

- Merge adjacent blocks.

Before

```
body:

  call void @foo(i32 noundef 0)
  %inc = add nuw nsw i32 0, 1
  %cmp = icmp slt i32 %inc, 2
  br label %body.1

body.1:
  call void @foo(i32 noundef %inc)
  %inc.1 = add nuw nsw i32 %inc, 1
  %cmp.1 = icmp slt i32 %inc.1, 2
  br label %exit

exit:
  ret void
```

After

```
body:
  call void @foo(i32 noundef 0)
  %inc = add nuw nsw i32 0, 1
  %cmp = icmp slt i32 %inc, 2
  call void @foo(i32 noundef %inc)
  %inc.1 = add nuw nsw i32 %inc, 1
  %cmp.1 = icmp slt i32 %inc.1, 2
  ret void
```

Cleaning up the Unrolled Code

- Merge adjacent blocks.
- Propagate constants.

Before

```
body:
  call void @foo(i32 noundef 0)
  %inc = add nuw nsw i32 0, 1
  %cmp = icmp slt i32 %inc, 2
  call void @foo(i32 noundef %inc)
  %inc.1 = add nuw nsw i32 %inc, 1
  %cmp.1 = icmp slt i32 %inc.1, 2
  ret void
```

After

```
body:
  call void @foo(i32 noundef 0)
  %inc = add nuw nsw i32 0, 1
  %cmp = icmp slt i32 1, 2
  call void @foo(i32 noundef 1)
  %inc.1 = add nuw nsw i32 1, 1
  %cmp.1 = icmp slt i32 2, 2
  ret void
```

Cleaning up the Unrolled Code

- Merge adjacent blocks.
- Propagate constants.
- Delete dead increments and comparisons.

Before

```
body:
  call void @foo(i32 noundef 0)
  %inc = add nuw nsw i32 0, 1
  %cmp = icmp slt i32 1, 2
  call void @foo(i32 noundef 1)
  %inc.1 = add nuw nsw i32 1, 1
  %cmp.1 = icmp slt i32 2, 2
  ret void
```

After

```
body:
  call void @foo(i32 noundef 0)
  call void @foo(i32 noundef 1)
  ret void
```

Unrolling the Loop: Recap

- Copy the loop body for the second iteration.
- Fix the copied values and PHIs, connect the iterations, and remove the backedge.
- **LoopUnroll itself** merges blocks, propagates constants, and deletes dead code.

Source

```
for (int i = 0; i < 2; ++i)
  foo(i);
```

After LoopUnroll

```
body:
  call void @foo(i32 noundef 0)
  call void @foo(i32 noundef 1)
  ret void
```

Full Unrolling

- Full unrolling **removes the backedge**.
- It creates **one copy of the body for every iteration**: `TripCount` copies.

Source

```
for (int i = 0; i < 2; ++i)
  foo(i);
```

After LoopUnroll

```
body:
  call void @foo(i32 noundef 0)
  call void @foo(i32 noundef 1)
  ret void
```

Unrolling a Larger Loop

- The same source, with a larger trip count: **100 instead of 2**.
- We could fully unroll it, but 100 copies would significantly increase code size.
- What if we **partially unroll by a factor of 2**?

Source

```
for (int i = 0; i < 100; ++i)
  foo(i);
```

IR

```
body:
  %i = phi i32 [ 0, %entry ], [ %inc, %body ]
  call void @foo(i32 noundef %i)
  %inc = add nuw nsw i32 %i, 1
  %cmp = icmp slt i32 %inc, 100
  br i1 %cmp, label %body, label %exit

exit:
  ret void
```

Copying the Loop Body

× The PHIs and copied instructions still refer to the original loop.

Copied bodies (incomplete)

```
body:
  %i = phi i32 [ 0, %entry ], [ %inc, %body ]
  call void @foo(i32 noundef %i)
  %inc = add nuw nsw i32 %i, 1
  %cmp = icmp slt i32 %inc, 100
  br i1 %cmp, label %body, label %exit

body.1:
  %i.1 = phi i32 [ 0, %entry ], [ %inc, %body ]
  call void @foo(i32 noundef %i)
  %inc.1 = add nuw nsw i32 %i, 1
  %cmp.1 = icmp slt i32 %inc, 100
  br i1 %cmp, label %body, label %exit

exit:
  ret void
```

Updating the Copied Values

✓ Use `%inc` in the second body and feed `%inc.1` into the retained header PHI.

Copied bodies (incomplete)

```
body:
  %i = phi i32 [ 0, %entry ], [ %inc, %body ]
  call void @foo(i32 noundef %i)
  %inc = add nuw nsw i32 %i, 1
  %cmp = icmp slt i32 %inc, 100
  br i1 %cmp, label %body, label %exit

body.1:
  %i.1 = phi i32 [ 0, %entry ], [ %inc, %body ]
  call void @foo(i32 noundef %i)
  %inc.1 = add nuw nsw i32 %i, 1
  %cmp.1 = icmp slt i32 %inc, 100
  br i1 %cmp, label %body, label %exit

exit:
  ret void
```

Updated values (incomplete)

```
body:
  %i = phi i32 [ 0, %entry ], [ %inc.1, %body.1 ]
  call void @foo(i32 noundef %i)
  %inc = add nuw nsw i32 %i, 1
  %cmp = icmp slt i32 %inc, 100
  br i1 %cmp, label %body, label %exit

body.1:

  call void @foo(i32 noundef %inc)
  %inc.1 = add nuw nsw i32 %inc, 1
  %cmp.1 = icmp slt i32 %inc.1, 100
  br i1 %cmp.1, label %body.1, label %exit

exit:
  ret void
```

Connecting the Iterations

- ✓ Use `%inc` in the second body and feed `%inc.1` into the retained header PHI.
- × The branches still return to their own bodies.

Updated values (incomplete)

```
body:
  %i = phi i32 [ 0, %entry ], [ %inc.1, %body.1 ]
  call void @foo(i32 noundef %i)
  %inc = add nuw nsw i32 %i, 1
  %cmp = icmp slt i32 %inc, 100
  br i1 %cmp, label %body, label %exit

body.1:

  call void @foo(i32 noundef %inc)
  %inc.1 = add nuw nsw i32 %inc, 1
  %cmp.1 = icmp slt i32 %inc.1, 100
  br i1 %cmp.1, label %body.1, label %exit

exit:
  ret void
```

Connecting the Iterations

- ✓ Use `%inc` in the second body and feed `%inc.1` into the retained header PHI.
- ✓ Connect each latch to the next header, including the last back to the first.
- ✓ Retain a backedge: unroll factor **2** is smaller than the known trip count of **100**.

Updated values (incomplete)

```
body:
  %i = phi i32 [ 0, %entry ], [ %inc.1, %body.1 ]
  call void @foo(i32 noundef %i)
  %inc = add nuw nsw i32 %i, 1
  %cmp = icmp slt i32 %inc, 100
  br i1 %cmp, label %body, label %exit

body.1:

  call void @foo(i32 noundef %inc)
  %inc.1 = add nuw nsw i32 %inc, 1
  %cmp.1 = icmp slt i32 %inc.1, 100
  br i1 %cmp.1, label %body.1, label %exit

exit:
  ret void
```

Connected iterations

```
body:
  %i = phi i32 [ 0, %entry ], [ %inc.1, %body.1 ]
  call void @foo(i32 noundef %i)
  %inc = add nuw nsw i32 %i, 1
  %cmp = icmp slt i32 %inc, 100
  br i1 %cmp, label %body.1, label %exit

body.1:

  call void @foo(i32 noundef %inc)
  %inc.1 = add nuw nsw i32 %inc, 1
  %cmp.1 = icmp slt i32 %inc.1, 100
  br i1 %cmp.1, label %body, label %exit

exit:
  ret void
```

Cleaning up the Unrolled Code

- The first exit check is redundant, and the two bodies are still separate.

Before LoopUnroll cleanup

```
body:
  %i = phi i32 [ 0, %entry ], [ %inc.1, %body.1 ]
  call void @foo(i32 noundef %i)
  %inc = add nuw nsw i32 %i, 1
  %cmp = icmp slt i32 %inc, 100
  br i1 %cmp, label %body.1, label %exit

body.1:

  call void @foo(i32 noundef %inc)
  %inc.1 = add nuw nsw i32 %inc, 1
  %cmp.1 = icmp slt i32 %inc.1, 100
  br i1 %cmp.1, label %body, label %exit

exit:
  ret void
```

Cleaning up the Unrolled Code

- Fold the first exit check and merge the bodies; keep the final backedge.

Before LoopUnroll cleanup

```
body:
  %i = phi i32 [ 0, %entry ], [ %inc.1, %body.1 ]
  call void @foo(i32 noundef %i)
  %inc = add nuw nsw i32 %i, 1
  %cmp = icmp slt i32 %inc, 100
  br i1 %cmp, label %body.1, label %exit

body.1:

  call void @foo(i32 noundef %inc)
  %inc.1 = add nuw nsw i32 %inc, 1
  %cmp.1 = icmp slt i32 %inc.1, 100
  br i1 %cmp.1, label %body, label %exit

exit:
  ret void
```

After LoopUnroll cleanup

```
body:
  %i = phi i32 [ 0, %entry ], [ %inc.1, %body ]
  call void @foo(i32 noundef %i)
  %inc = add nuw nsw i32 %i, 1
  call void @foo(i32 noundef %inc)
  %inc.1 = add nuw nsw i32 %i, 2
  %cmp.1 = icmp slt i32 %inc.1, 100
  br i1 %cmp.1, label %body, label %exit

exit:
  ret void
```

Partial Unrolling

- Partial unrolling **keeps the backedge** and creates copies equal to the **selected unroll factor**.
- Here, factor **2** is smaller than trip count **100**; each group advances `i` by two.

Source

```
for (int i = 0; i < 100; ++i)
  foo(i);
```

After LoopUnroll

```
body:
  %i = phi i32 [ 0, %entry ], [ %inc.1, %body ]
  call void @foo(i32 noundef %i)
  %inc = add nuw nsw i32 %i, 1
  call void @foo(i32 noundef %inc)
  %inc.1 = add nuw nsw i32 %i, 2
  %cmp.1 = icmp slt i32 %inc.1, 100
  br i1 %cmp.1, label %body, label %exit

exit:
  ret void
```

Full and Partial Unrolling Comparison

Form	Loop backedge	Copies of the body
Full unrolling	Removed	One copy for every iteration
Partial unrolling	Retained	Copies equal to the selected unroll factor

- Partial unrolling differs from full unrolling in that:
  - It uses fewer bodies than the trip count and keeps the loop.

A Loop with at Most Three Iterations

- What about a loop whose trip count is unknown, but whose upper bound is known?

Source

```
n = min(n, 3);
for (int i = 0; i < n; ++i)
  foo(i);
```

Recall our trip counts:

- Exact compile-time count
- **Known upper bound**
- Runtime count

IR

```
entry:
  %bound = call i32 @llvm.smin.i32(i32 %n, i32 3)
  %enter = icmp slt i32 0, %bound
  br i1 %enter, label %preheader, label %exit
preheader:
  br label %body
body:
  %i = phi i32 [ 0, %preheader ], [ %inc, %body ]
  call void @foo(i32 noundef %i)
  %inc = add nuw nsw i32 %i, 1
  %cmp = icmp slt i32 %inc, %bound
  br i1 %cmp, label %body, label %loop.exit
loop.exit:
  br label %exit
exit:
  ret void
```

A Loop with at Most Three Iterations

- What about a loop whose trip count is unknown, but whose upper bound is known?

Source

```
n = min(n, 3);
for (int i = 0; i < n; ++i)
  foo(i);
```

Recall our trip counts:

- Exact compile-time count
- **Known upper bound**
- Runtime count

IR

```
entry:
  %bound = call i32 @llvm.smin.i32(i32 %n, i32 3)
  %enter = icmp slt i32 0, %bound
  br i1 %enter, label %preheader, label %exit
preheader:
  br label %body
body:
  %i = phi i32 [ 0, %preheader ], [ %inc, %body ]
  call void @foo(i32 noundef %i)
  %inc = add nuw nsw i32 %i, 1
  %cmp = icmp slt i32 %inc, %bound
  br i1 %cmp, label %body, label %loop.exit
loop.exit:
  br label %exit
exit:
  ret void
```

A Loop with at Most Three Iterations

- What about a loop whose trip count is unknown, but whose upper bound is known?
- How can we remove the loop structure here?
  - We know there are at most three iterations, so we can create **three copies**.

Source

```
n = min(n, 3);
for (int i = 0; i < n; ++i)
  foo(i);
```

Recall our trip counts:

- Exact compile-time count
- **Known upper bound**
- Runtime count

IR

```
entry:
  %bound = call i32 @llvm.smin.i32(i32 %n, i32 3)
  %enter = icmp slt i32 0, %bound
  br i1 %enter, label %preheader, label %exit
preheader:
  br label %body
body:
  %i = phi i32 [ 0, %preheader ], [ %inc, %body ]
  call void @foo(i32 noundef %i)
  %inc = add nuw nsw i32 %i, 1
  %cmp = icmp slt i32 %inc, %bound
  br i1 %cmp, label %body, label %loop.exit
loop.exit:
  br label %exit
exit:
  ret void
```

Three Copies for Three Possible Iterations

Before LoopUnroll

```
entry:
  %bound = call i32 @llvm.smin.i32(i32 %n, i32 3)
  %enter = icmp slt i32 0, %bound
  br i1 %enter, label %preheader, label %exit
preheader:
  br label %body
body:
  %i = phi i32 [ 0, %preheader ], [ %inc, %body ]
  call void @foo(i32 noundef %i)
  %inc = add nuw nsw i32 %i, 1
  %cmp = icmp slt i32 %inc, %bound
  br i1 %cmp, label %body, label %loop.exit
loop.exit:
  br label %exit
exit:
  ret void
```

- Make one copy for each possible iteration: `foo(0)`, `foo(1)`, `foo(2)`.

After LoopUnroll

```
entry:
  %bound = call i32 @llvm.smin.i32(i32 %n, i32 3)
  %enter = icmp slt i32 0, %bound
  br i1 %enter, label %preheader, label %exit
preheader:
  br label %body
body:
  call void @foo(i32 noundef 0)
  %cmp = icmp slt i32 1, %bound
  br i1 %cmp, label %body.1, label %loop.exit
body.1:
  call void @foo(i32 noundef 1)
  %cmp.1 = icmp slt i32 2, %bound
  br i1 %cmp.1, label %body.2, label %loop.exit
body.2:
  call void @foo(i32 noundef 2)
  br label %loop.exit
loop.exit:
  br label %exit
exit:
  ret void
```

Checks Between the Copies

Before LoopUnroll

```
entry:
  %bound = call i32 @llvm.smin.i32(i32 %n, i32 3)
  %enter = icmp slt i32 0, %bound
  br i1 %enter, label %preheader, label %exit
preheader:
  br label %body
body:
  %i = phi i32 [ 0, %preheader ], [ %inc, %body ]
  call void @foo(i32 noundef %i)
  %inc = add nuw nsw i32 %i, 1
  %cmp = icmp slt i32 %inc, %bound
  br i1 %cmp, label %body, label %loop.exit
loop.exit:
  br label %exit
exit:
  ret void
```

- Keep a check before each possible iteration: `0 < bound`, `1 < bound`, `2 < bound`.

After LoopUnroll

```
entry:
  %bound = call i32 @llvm.smin.i32(i32 %n, i32 3)
  %enter = icmp slt i32 0, %bound
  br i1 %enter, label %preheader, label %exit
preheader:
  br label %body
body:
  call void @foo(i32 noundef 0)
  %cmp = icmp slt i32 1, %bound
  br i1 %cmp, label %body.1, label %loop.exit
body.1:
  call void @foo(i32 noundef 1)
  %cmp.1 = icmp slt i32 2, %bound
  br i1 %cmp.1, label %body.2, label %loop.exit
body.2:
  call void @foo(i32 noundef 2)
  br label %loop.exit
loop.exit:
  br label %exit
exit:
  ret void
```

Every Possible Iteration Has a Copy

Before LoopUnroll

```
entry:
  %bound = call i32 @llvm.smin.i32(i32 %n, i32 3)
  %enter = icmp slt i32 0, %bound
  br i1 %enter, label %preheader, label %exit
preheader:
  br label %body
body:
  %i = phi i32 [ 0, %preheader ], [ %inc, %body ]
  call void @foo(i32 noundef %i)
  %inc = add nuw nsw i32 %i, 1
  %cmp = icmp slt i32 %inc, %bound
  br i1 %cmp, label %body, label %loop.exit
loop.exit:
  br label %exit
exit:
  ret void
```

- Three copies cover every possible iteration. **No backedge remains.**

After LoopUnroll

```
entry:
  %bound = call i32 @llvm.smin.i32(i32 %n, i32 3)
  %enter = icmp slt i32 0, %bound
  br i1 %enter, label %preheader, label %exit
preheader:
  br label %body
body:
  call void @foo(i32 noundef 0)
  %cmp = icmp slt i32 1, %bound
  br i1 %cmp, label %body.1, label %loop.exit
body.1:
  call void @foo(i32 noundef 1)
  %cmp.1 = icmp slt i32 2, %bound
  br i1 %cmp.1, label %body.2, label %loop.exit
body.2:
  call void @foo(i32 noundef 2)
  br label %loop.exit
loop.exit:
  br label %exit
exit:
  ret void
```

Upper-Bound Unrolling

Source

```
n = min(n, 3);
for (int i = 0; i < n; ++i)
  foo(i);
```

- **One copy per possible iteration**; **no backedge**.
- Exit checks skip unneeded iterations.

After LoopUnroll

```
entry:
  %bound = call i32 @llvm.smin.i32(i32 %n, i32 3)
  %enter = icmp slt i32 0, %bound
  br i1 %enter, label %preheader, label %exit
preheader:
  br label %body
body:
  call void @foo(i32 noundef 0)
  %cmp = icmp slt i32 1, %bound
  br i1 %cmp, label %body.1, label %loop.exit
body.1:
  call void @foo(i32 noundef 1)
  %cmp.1 = icmp slt i32 2, %bound
  br i1 %cmp.1, label %body.2, label %loop.exit
body.2:
  call void @foo(i32 noundef 2)
  br label %loop.exit
loop.exit:
  br label %exit
exit:
  ret void
```

Full, Partial, and Upper-Bound Unrolling

Form	Loop backedge	Trip count	Copies of the body	Trip-count checks
Full	Removed	Exact compile-time count	One copy for every iteration	Fold away
Partial	Retained	Exact compile-time count	Copies equal to the selected factor	Retained to end the loop
Upper-bound	Removed	Unknown; compile-time upper bound	One copy for every possible iteration	Entry guard and checks between copies

- Upper-bound unrolling removes the backedge but keeps checks for iterations that may not execute.

A Trip Count Known at Runtime

Source

```
for (int i = 0; i < n; ++i)
  foo(i);
```

IR

```
entry:
  %nonempty = icmp slt i32 0, %n
  br i1 %nonempty, label %preheader, label %return

preheader:
  br label %body

body:
  %i = phi i32 [ 0, %preheader ], [ %inc, %body ]
  call void @foo(i32 noundef %i)
  %inc = add nuw nsw i32 %i, 1
  %cmp = icmp slt i32 %inc, %n
  br i1 %cmp, label %body, label %exit

exit:
  br label %return

return:
  ret void
```

A Trip Count Known at Runtime

Source

```
for (int i = 0; i < n; ++i)
  foo(i);
```

- Can we remove the loop structure here?

IR

```
entry:
  %nonempty = icmp slt i32 0, %n
  br i1 %nonempty, label %preheader, label %return

preheader:
  br label %body

body:
  %i = phi i32 [ 0, %preheader ], [ %inc, %body ]
  call void @foo(i32 noundef %i)
  %inc = add nuw nsw i32 %i, 1
  %cmp = icmp slt i32 %inc, %n
  br i1 %cmp, label %body, label %exit

exit:
  br label %return

return:
  ret void
```

A Trip Count Known at Runtime

Source

```
for (int i = 0; i < n; ++i)
  foo(i);
```

- Can we remove the loop structure here?
  - No: we know neither the trip count nor a useful upper bound at compile time.

IR

```
entry:
  %nonempty = icmp slt i32 0, %n
  br i1 %nonempty, label %preheader, label %return

preheader:
  br label %body

body:
  %i = phi i32 [ 0, %preheader ], [ %inc, %body ]
  call void @foo(i32 noundef %i)
  %inc = add nuw nsw i32 %i, 1
  %cmp = icmp slt i32 %inc, %n
  br i1 %cmp, label %body, label %exit

exit:
  br label %return

return:
  ret void
```

A Trip Count Known at Runtime

Source

```
for (int i = 0; i < n; ++i)
  foo(i);
```

- Can we remove the loop structure here?
  - No: we know neither the trip count nor a useful upper bound at compile time.
- How can we unroll by a factor of **4**?

IR

```
entry:
  %nonempty = icmp slt i32 0, %n
  br i1 %nonempty, label %preheader, label %return

preheader:
  br label %body

body:
  %i = phi i32 [ 0, %preheader ], [ %inc, %body ]
  call void @foo(i32 noundef %i)
  %inc = add nuw nsw i32 %i, 1
  %cmp = icmp slt i32 %inc, %n
  br i1 %cmp, label %body, label %exit

exit:
  br label %return

return:
  ret void
```

Unrolling the Main Loop

Rolled loop (excerpt)

```
body:
  %i = phi i32 [ 0, %preheader ], [ %inc, %body ]
  call void @foo(i32 noundef %i)
  %inc = add nuw nsw i32 %i, 1
  %cmp = icmp slt i32 %inc, %n
  br i1 %cmp, label %body, label %exit
```

Four copied bodies (incomplete)

```
body:
  %i = phi i32 [ 0, %preheader ], [ %inc.3, %body ]
  call void @foo(i32 noundef %i)
  %inc = add i32 %i, 1
  call void @foo(i32 noundef %inc)
  %inc.1 = add i32 %i, 2
  call void @foo(i32 noundef %inc.1)
  %inc.2 = add i32 %i, 3
  call void @foo(i32 noundef %inc.2)
  %inc.3 = add i32 %i, 4
  %cmp = icmp slt i32 %inc.3, %n
  br i1 %cmp, label %body, label %exit
```

Unrolling the Main Loop

× The trip count may not be divisible by four, but this attempt checks only after four calls.
  - For `n = 10`, the last group calls `foo(8)`, `foo(9)`, `foo(10)`, and `foo(11)`. The last two calls must not execute.

Rolled loop (excerpt)

```
body:
  %i = phi i32 [ 0, %preheader ], [ %inc, %body ]
  call void @foo(i32 noundef %i)
  %inc = add nuw nsw i32 %i, 1
  %cmp = icmp slt i32 %inc, %n
  br i1 %cmp, label %body, label %exit
```

Four copied bodies (incomplete)

```
body:
  %i = phi i32 [ 0, %preheader ], [ %inc.3, %body ]
  call void @foo(i32 noundef %i)
  %inc = add i32 %i, 1
  call void @foo(i32 noundef %inc)
  %inc.1 = add i32 %i, 2
  call void @foo(i32 noundef %inc.1)
  %inc.2 = add i32 %i, 3
  call void @foo(i32 noundef %inc.2)
  %inc.3 = add i32 %i, 4
  %cmp = icmp slt i32 %inc.3, %n
  br i1 %cmp, label %body, label %exit
```

Handling the Leftover Iterations

✓ Run complete groups of four in the main loop; use a remainder loop for the final 0–3 iterations.

Final IR excerpts

Main loop: groups of four

```
body:
  %i = phi i32 [ 0, %main.pre ], [ %inc.3, %body ]
  %iter = phi i32 [ 0, %main.pre ], [ %next.3, %body ]
  call void @foo(i32 noundef %i)
  %inc = add nuw nsw i32 %i, 1
  call void @foo(i32 noundef %inc)
  %inc.1 = add nuw nsw i32 %i, 2
  call void @foo(i32 noundef %inc.1)
  %inc.2 = add nuw nsw i32 %i, 3
  call void @foo(i32 noundef %inc.2)
  %inc.3 = add nuw nsw i32 %i, 4
  %next.3 = add nuw nsw i32 %iter, 4
  %more.3 = icmp slt i32 %next.3, %main.count
  br i1 %more.3, label %body, label %main.exit
```

Remainder loop: leftover iterations

```
epil:
  %i.epil = phi i32 [ %i.start, %epil.pre ], [ %inc.epil, %epil ]
  %epil.iter = phi i32 [ 0, %epil.pre ], [ %epil.next, %epil ]
  call void @foo(i32 noundef %i.epil)
  %inc.epil = add nuw nsw i32 %i.epil, 1
  %cmp.epil = icmp slt i32 %inc.epil, %n
  %epil.next = add i32 %epil.iter, 1
  %epil.more = icmp slt i32 %epil.next, %extra
  br i1 %epil.more, label %epil, label %epil.exit
```

Connecting the Loops

- How do we choose the main-loop limit and the remainder's starting index?

Final IR excerpts

Main loop: groups of four

```
body:
  %i = phi i32 [ 0, %main.pre ], [ %inc.3, %body ]
  %iter = phi i32 [ 0, %main.pre ], [ %next.3, %body ]
  call void @foo(i32 noundef %i)
  %inc = add nuw nsw i32 %i, 1
  call void @foo(i32 noundef %inc)
  %inc.1 = add nuw nsw i32 %i, 2
  call void @foo(i32 noundef %inc.1)
  %inc.2 = add nuw nsw i32 %i, 3
  call void @foo(i32 noundef %inc.2)
  %inc.3 = add nuw nsw i32 %i, 4
  %next.3 = add nuw nsw i32 %iter, 4
  %more.3 = icmp slt i32 %next.3, %main.count
  br i1 %more.3, label %body, label %main.exit
```

Remainder loop: leftover iterations

```
epil:
  %i.epil = phi i32 [ %i.start, %epil.pre ], [ %inc.epil, %epil ]
  %epil.iter = phi i32 [ 0, %epil.pre ], [ %epil.next, %epil ]
  call void @foo(i32 noundef %i.epil)
  %inc.epil = add nuw nsw i32 %i.epil, 1
  %cmp.epil = icmp slt i32 %inc.epil, %n
  %epil.next = add i32 %epil.iter, 1
  %epil.more = icmp slt i32 %epil.next, %extra
  br i1 %epil.more, label %epil, label %epil.exit
```

Connecting the Loops

- ✓ Compute how many iterations belong to each loop.
  - `extra = n % 4`: the remainder loop executes `extra` times.
  - `main.count = n - extra`: the main loop executes `main.count / 4` times.

Main loop (excerpt)

```
body:
  %i = phi i32 [ 0, %main.pre ], [ %inc.3, %body ]
  %iter = phi i32 [ 0, %main.pre ], [ %next.3, %body ]
  call void @foo(i32 noundef %i)
  %inc = add nuw nsw i32 %i, 1
  call void @foo(i32 noundef %inc)
  %inc.1 = add nuw nsw i32 %i, 2
  call void @foo(i32 noundef %inc.1)
  %inc.2 = add nuw nsw i32 %i, 3
  call void @foo(i32 noundef %inc.2)
  %inc.3 = add nuw nsw i32 %i, 4
  %next.3 = add nuw nsw i32 %iter, 4
  %more.3 = icmp slt i32 %next.3, %main.count
  br i1 %more.3, label %body, label %main.exit
```

Entry and preheaders (excerpt)

```
entry:
  %nonempty = icmp slt i32 0, %n
  br i1 %nonempty, label %preheader, label %return
preheader:
  %last = add i32 %n, -1
  %extra = and i32 %n, 3
  %small = icmp slt i32 %n, 4
  br i1 %small, label %epil.pre, label %main.pre
main.pre:
  %main.count = sub i32 %n, %extra
  br label %body
```

Connecting the Loops

- ✓ Compute how many iterations belong to each loop.
  - `extra = n % 4`: the remainder loop executes `extra` times.
  - `main.count = n - extra`: the main loop executes `main.count / 4` times.
- ✓ Keep the entry guard; bypass the main loop when `n < 4`.
  - `n <= 0`: return without any calls.
  - `1 <= n < 4`: go straight to the remainder.
  - `n >= 4`: go straight to the main-loop body.

Main loop (excerpt)

```
body:
  %i = phi i32 [ 0, %main.pre ], [ %inc.3, %body ]
  %iter = phi i32 [ 0, %main.pre ], [ %next.3, %body ]
  call void @foo(i32 noundef %i)
  %inc = add nuw nsw i32 %i, 1
  call void @foo(i32 noundef %inc)
  %inc.1 = add nuw nsw i32 %i, 2
  call void @foo(i32 noundef %inc.1)
  %inc.2 = add nuw nsw i32 %i, 3
  call void @foo(i32 noundef %inc.2)
  %inc.3 = add nuw nsw i32 %i, 4
  %next.3 = add nuw nsw i32 %iter, 4
  %more.3 = icmp slt i32 %next.3, %main.count
  br i1 %more.3, label %body, label %main.exit
```

Entry and preheaders (excerpt)

```
entry:
  %nonempty = icmp slt i32 0, %n
  br i1 %nonempty, label %preheader, label %return
preheader:
  %last = add i32 %n, -1
  %extra = and i32 %n, 3
  %small = icmp slt i32 %n, 4
  br i1 %small, label %epil.pre, label %main.pre
main.pre:
  %main.count = sub i32 %n, %extra
  br label %body
```

Runtime Unrolling: The Complete Expansion

Entry, setup, and main loop

```
entry:
  %nonempty = icmp slt i32 0, %n
  br i1 %nonempty, label %preheader, label %return

preheader:
  %last = add i32 %n, -1
  %extra = and i32 %n, 3
  %small = icmp slt i32 %n, 4
  br i1 %small, label %epil.pre, label %main.pre

main.pre:
  %main.count = sub i32 %n, %extra
  br label %body

body:
  %i = phi i32 [ 0, %main.pre ], [ %inc.3, %body ]
  %iter = phi i32 [ 0, %main.pre ], [ %next.3, %body ]
  call void @foo(i32 noundef %i)
  %inc = add nuw nsw i32 %i, 1
  call void @foo(i32 noundef %inc)
  %inc.1 = add nuw nsw i32 %i, 2
  call void @foo(i32 noundef %inc.1)
  %inc.2 = add nuw nsw i32 %i, 3
  call void @foo(i32 noundef %inc.2)
  %inc.3 = add nuw nsw i32 %i, 4
  %next.3 = add nuw nsw i32 %iter, 4
  %more.3 = icmp slt i32 %next.3, %main.count
  br i1 %more.3, label %body, label %main.exit
```

Remainder and exits

```
main.exit:
  %i.unr = phi i32 [ %inc.3, %body ]
  %has.extra = icmp sgt i32 %extra, 0
  br i1 %has.extra, label %epil.pre, label %exit

epil.pre:
  %i.start = phi i32 [ 0, %preheader ], [ %i.unr, %main.exit ]
  %has.extra1 = icmp sgt i32 %extra, 0
  call void @llvm.assume(i1 %has.extra1)
  br label %epil

epil:
  %i.epil = phi i32 [ %i.start, %epil.pre ], [ %inc.epil, %epil ]
  %epil.iter = phi i32 [ 0, %epil.pre ], [ %epil.next, %epil ]
  call void @foo(i32 noundef %i.epil)
  %inc.epil = add nuw nsw i32 %i.epil, 1
  %cmp.epil = icmp slt i32 %inc.epil, %n
  %epil.next = add i32 %epil.iter, 1
  %epil.more = icmp slt i32 %epil.next, %extra
  br i1 %epil.more, label %epil, label %epil.exit

epil.exit:
  br label %exit

exit:
  br label %return

return:
  ret void
```

Runtime Unrolling

Source

```
for (int i = 0; i < n; ++i)
  foo(i);
```

For positive `n`:

```
extra = n % 4
main.count = n - extra
```

- Runtime unrolling uses a trip count **computed at runtime**.
- The main loop **keeps its backedge**; a **remainder loop** handles up to three leftovers.

After LoopUnroll (excerpts)

Main loop: four calls per group

```
body:
  %i = phi i32 [ 0, %main.pre ], [ %inc.3, %body ]
  %iter = phi i32 [ 0, %main.pre ], [ %next.3, %body ]
  call void @foo(i32 noundef %i)
  %inc = add nuw nsw i32 %i, 1
  call void @foo(i32 noundef %inc)
  %inc.1 = add nuw nsw i32 %i, 2
  call void @foo(i32 noundef %inc.1)
  %inc.2 = add nuw nsw i32 %i, 3
  call void @foo(i32 noundef %inc.2)
  %inc.3 = add nuw nsw i32 %i, 4
  %next.3 = add nuw nsw i32 %iter, 4
  %more.3 = icmp slt i32 %next.3, %main.count
  br i1 %more.3, label %body, label %main.exit
```

Remainder loop: one call per iteration

```
epil:
  %i.epil = phi i32 [ %i.start, %epil.pre ], [ %inc.epil, %epil ]
  %epil.iter = phi i32 [ 0, %epil.pre ], [ %epil.next, %epil ]
  call void @foo(i32 noundef %i.epil)
  %inc.epil = add nuw nsw i32 %i.epil, 1
  %cmp.epil = icmp slt i32 %inc.epil, %n
  %epil.next = add i32 %epil.iter, 1
  %epil.more = icmp slt i32 %epil.next, %extra
  br i1 %epil.more, label %epil, label %epil.exit
```

Comparing the Four Forms

Form	Loop backedge	Trip count	Copies of the body	Trip-count checks	Remainder
Full	Removed	Exact compile-time count	One copy for every iteration	Fold away	No
Partial	Retained	Exact compile-time count	Copies equal to the selected factor	Retained to end the loop	No
Upper-bound	Removed	Unknown; compile-time upper bound	One copy for every possible iteration	Entry guard and checks between copies	No
Runtime	Main loop retained	Computed at runtime	Selected factor in main loop; one remainder copy	Guards; one test per group and per remainder iteration	Yes

Table of Contents

Background

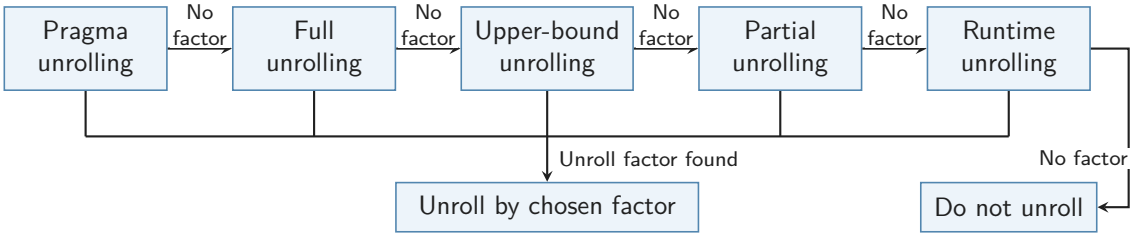
Unrolling Different Kinds of Loops

Deciding to Unroll

Pass Interactions at -O3

Notes for the Road

Unrolling Decision Order



- Consider the associated heuristics for each kind of unrolling in priority order.
- Try the next applicable category only if no factor was selected.
- Once a candidate factor is selected, stop searching and attempt to unroll by that factor.

Target Unrolling Preferences

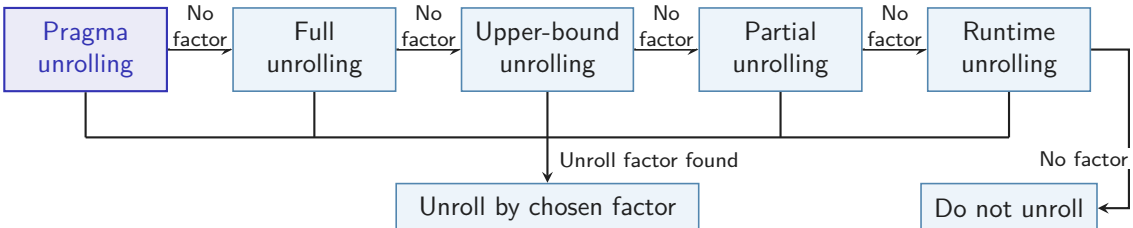
`UP` holds the unroller's `UnrollingPreferences`.

LLVM calls `TTI.getUnrollingPreferences(L, ...)` for each loop `L`.

Targets can tailor these settings to that loop, such as enabling upper-bound unrolling only for selected loop shapes.

- **Enable forms by default**: `UP.UpperBound`, `UP.Partial`, and `UP.Runtime`.
- **Tune size budgets**: `UP.Threshold` (default: 300) and `UP.PartialThreshold` (default: 150).
- **Choose factors and limits**: `UP.DefaultUnrollRuntimeCount` (default: 8) and `UP.MaxUpperBound` (default: 8).

Unrolling Decision Order



- Pragma unrolling tries to honor the programmer's preferences for how to unroll a particular loop.

Pragmas and Loop Metadata

Pragmas reach the unroller as `!llvm.loop` metadata.

Source

```
#pragma nounroll
for (int i = 0; i < 4; ++i)
  foo(i);
```

The metadata is attached to the **branch instruction that takes the backedge**.

LLVM IR

```
loop:
  %i = phi i32 [ 0, %entry ], [ %next, %loop ]
  call void @foo(i32 %i)
  %next = add nuw nsw i32 %i, 1
  %more = icmp ult i32 %next, 4
  br i1 %more, label %loop, label %exit, !llvm.loop !0

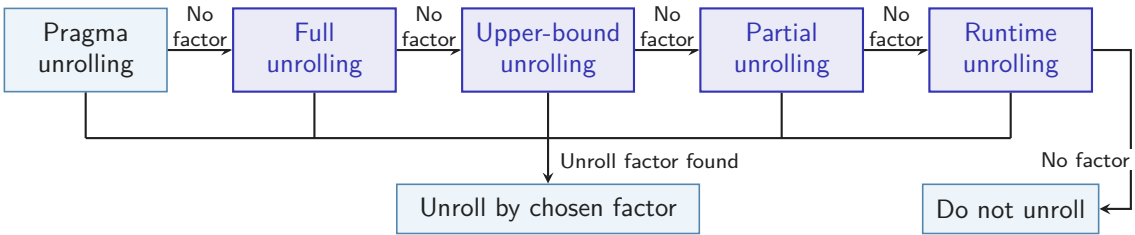
!0 = distinct !{!0, !1}
!1 = !{!"llvm.loop.unroll.disable"}
```

Unrolling Pragmas

C++ pragma	LLVM metadata name	Impact
`#pragma nounroll`	`llvm.loop.unroll.disable`	Disable unrolling.
`#pragma clang loop unroll(full)`	`llvm.loop.unroll.full`	Request full unrolling.
`#pragma unroll(N)`	`llvm.loop.unroll.count`	Partial or runtime unroll by a factor of `N`.
`#pragma unroll`	`llvm.loop.unroll.enable`	Boost unrolling thresholds.

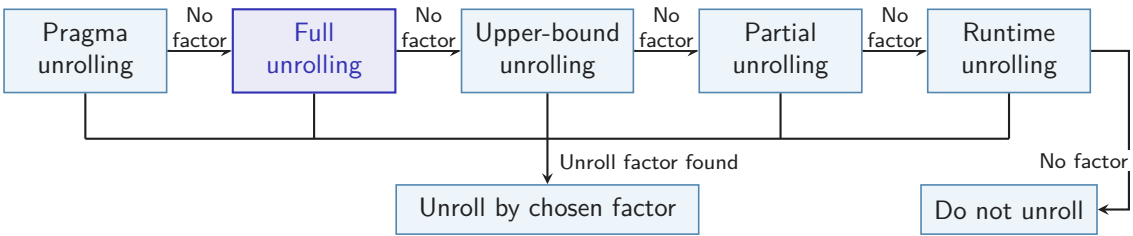
Unrolling Decision Order

Now focus on the compiler's cost-based choices.



- The shared starting point is **estimated unrolled code size**.

Unrolling Decision Order



Full Unrolling Heuristics

Example

```
for (int i = 0; i < 8; ++i) {
  foo(i); foo(i); foo(i);
  foo(i); foo(i); foo(i);
  foo(i); foo(i); foo(i);
  foo(i); foo(i); foo(i);
}
```

Trip count: **8**

Rolled loop size: **27** cost units

Full Unrolling Heuristics

Trip-Count Requirements

- Trip count is a compile-time constant.
  - ✓ Trip count: **8**.

Example

```
for (int i = 0; i < 8; ++i) {
  foo(i); foo(i); foo(i);
  foo(i); foo(i); foo(i);
  foo(i); foo(i); foo(i);
  foo(i); foo(i); foo(i);
}
```

Trip count: **8**

Rolled loop size: **27** cost units

Full Unrolling Heuristics

Trip-Count Requirements

- Trip count is a compile-time constant.
  - ✓ Trip count: **8**.

Heuristics

- `EstimatedUnrolledLoopSize(L, TripCount)` < `UP.Threshold` (default: 300).
  - ✓ Expansion fits the size limit.

Example

```
for (int i = 0; i < 8; ++i) {
  foo(i); foo(i); foo(i);
  foo(i); foo(i); foo(i);
  foo(i); foo(i); foo(i);
  foo(i); foo(i); foo(i);
}
```

Trip count: **8**

Rolled loop size: **27** cost units

Estimated unrolled loop size = rolled loop size × unroll factor
= 27 × 8
= **216**

Estimated unrolled loop size < `UP.Threshold`
216 < 300: Accept.

Full Unrolling Heuristics

Trip-Count Requirements

- Trip count is a compile-time constant.
  - ✓ Trip count: **8**.

Heuristics

- `EstimatedUnrolledLoopSize(L, TripCount)` < `UP.Threshold` (default: 300).
  - ✓ Expansion fits the size limit.

Example

```
for (int i = 0; i < 8; ++i) {
  foo(i); foo(i); foo(i);
  foo(i); foo(i); foo(i);
  foo(i); foo(i); foo(i);
  foo(i); foo(i); foo(i);
}
```

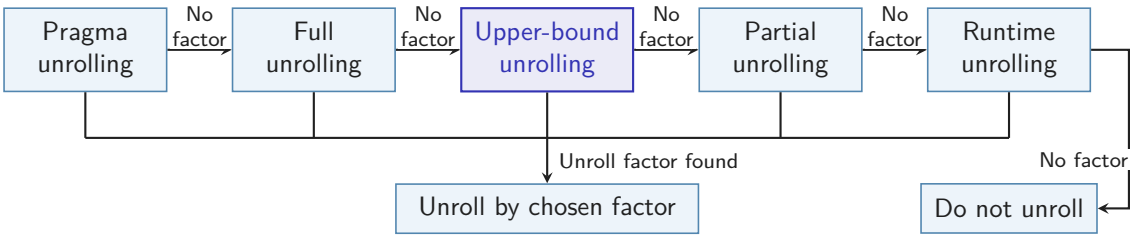
Trip count: **8**

Rolled loop size: **27** cost units

Conclusion

Fully unroll.

Unrolling Decision Order



Upper-Bound Unrolling Heuristics

Example

```
n = min(n, 8);
for (int i = 0; i < n; ++i) {
  foo(i); foo(i); foo(i); foo(i); foo(i);
  foo(i); foo(i); foo(i); foo(i); foo(i);
  foo(i); foo(i); foo(i); foo(i); foo(i);
  foo(i); foo(i); foo(i); foo(i); foo(i);
}
```

Trip count:	Unknown
Max trip count:	**8**
Rolled loop size:	**43** cost units

Upper-Bound Unrolling Heuristics

Trip-Count Requirements

- Trip count is not a compile-time constant.
  - ✓ Trip count depends on `n`.

Example

```
n = min(n, 8);
for (int i = 0; i < n; ++i) {
  foo(i); foo(i); foo(i); foo(i); foo(i);
  foo(i); foo(i); foo(i); foo(i); foo(i);
  foo(i); foo(i); foo(i); foo(i); foo(i);
  foo(i); foo(i); foo(i); foo(i); foo(i);
}
```

Trip count:	Unknown
Max trip count:	**8**
Rolled loop size:	**43** cost units

Upper-Bound Unrolling Heuristics

Trip-Count Requirements

- Trip count is not a compile-time constant.
  - ✓ Trip count depends on `n`.
- Max trip count is a compile-time constant.
  - ✓ Maximum: **8 iterations**.

Example

```
n = min(n, 8);
for (int i = 0; i < n; ++i) {
  foo(i); foo(i); foo(i); foo(i); foo(i);
  foo(i); foo(i); foo(i); foo(i); foo(i);
  foo(i); foo(i); foo(i); foo(i); foo(i);
  foo(i); foo(i); foo(i); foo(i); foo(i);
}
```

Trip count:	Unknown
Max trip count:	**8**
Rolled loop size:	**43** cost units

Upper-Bound Unrolling Heuristics

Trip-Count Requirements

- Trip count is not a compile-time constant.
  - ✓ Trip count depends on `n`.
- Max trip count is a compile-time constant.
  - ✓ Maximum: **8 iterations**.

Heuristics

- `MaxTripCount` $\leq$ `UP.MaxUpperBound` (default: 8).
  - ✓ Maximum fits the limit.

Example

```
n = min(n, 8);
for (int i = 0; i < n; ++i) {
  foo(i); foo(i); foo(i); foo(i); foo(i);
  foo(i); foo(i); foo(i); foo(i); foo(i);
  foo(i); foo(i); foo(i); foo(i); foo(i);
  foo(i); foo(i); foo(i); foo(i); foo(i);
}
```

Trip count:	Unknown
Max trip count:	**8**
Rolled loop size:	**43** cost units

Bound check

`MaxTripCount` $\leq$ `UP.MaxUpperBound`

$8 \leq 8$

Accept the bound.

Upper-Bound Unrolling Heuristics

Trip-Count Requirements

- Trip count is not a compile-time constant.
  - ✓ Trip count depends on `n`.
- Max trip count is a compile-time constant.
  - ✓ Maximum: **8 iterations**.

Heuristics

- `MaxTripCount` $\leq$ `UP.MaxUpperBound` (default: 8).
  - ✓ Maximum fits the limit.
- `EstimatedUnrolledLoopSize(L, MaxTripCount)` $<$ `UP.Threshold` (default: 300).
  - × Expansion exceeds the size limit.

Example

```
n = min(n, 8);
for (int i = 0; i < n; ++i) {
  foo(i); foo(i); foo(i); foo(i); foo(i);
  foo(i); foo(i); foo(i); foo(i); foo(i);
  foo(i); foo(i); foo(i); foo(i); foo(i);
  foo(i); foo(i); foo(i); foo(i); foo(i);
}
```

Trip count:	Unknown
Max trip count:	**8**
Rolled loop size:	**43** cost units

Estimated unrolled loop size $<$ `UP.Threshold`

rolled loop size $\times$ unroll factor $<$ `UP.Threshold`

$43 \times 8 < 300$

$344 < 300$ False: reject.

Upper-Bound Unrolling Heuristics

Trip-Count Requirements

- Trip count is not a compile-time constant.
  - ✓ Trip count depends on `n`.
- Max trip count is a compile-time constant.
  - ✓ Maximum: **8 iterations**.

Heuristics

- `MaxTripCount` $\leq$ `UP.MaxUpperBound` (default: 8).
  - ✓ Maximum fits the limit.
- `EstimatedUnrolledLoopSize(L, MaxTripCount)` $<$ `UP.Threshold` (default: 300).
  - × Expansion exceeds the size limit.

Example

```
n = min(n, 8);
for (int i = 0; i < n; ++i) {
  foo(i); foo(i); foo(i); foo(i); foo(i);
  foo(i); foo(i); foo(i); foo(i); foo(i);
  foo(i); foo(i); foo(i); foo(i); foo(i);
  foo(i); foo(i); foo(i); foo(i); foo(i);
}
```

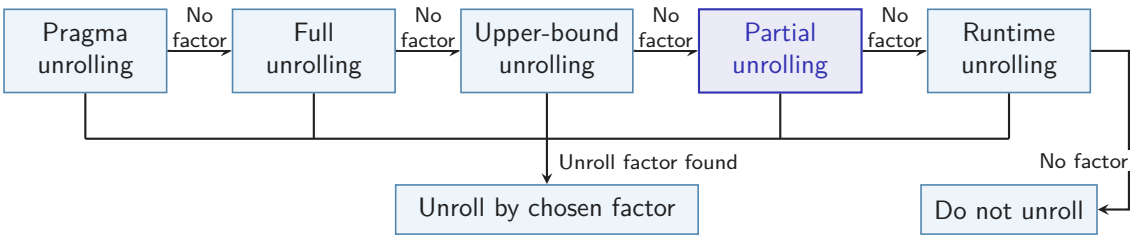
Trip count:	Unknown
Max trip count:	**8**
Rolled loop size:	**43** cost units

Conclusion

Reject upper-bound unrolling.

Try the next set of heuristics.

Unrolling Decision Order



Partial Unrolling Heuristics

Example

```
for (int i = 0; i < 12; ++i) {
  foo(i); foo(i); foo(i);
  foo(i); foo(i); foo(i);
  foo(i); foo(i); foo(i);
  foo(i); foo(i); foo(i);
}
```

Trip count: **12**

Rolled loop size: **27** cost units

Partial Unrolling Heuristics

Trip-Count Requirements

- Trip count is a compile-time constant.
  - ✓ Trip count: **12**.

Example

```
for (int i = 0; i < 12; ++i) {
  foo(i); foo(i); foo(i);
  foo(i); foo(i); foo(i);
  foo(i); foo(i); foo(i);
  foo(i); foo(i); foo(i);
}
```

Trip count: **12**

Rolled loop size: **27** cost units

Partial Unrolling Heuristics

Trip-Count Requirements

- Trip count is a compile-time constant.
  - ✓ Trip count: **12**.

Heuristics

- If **estimated unrolled loop size** exceeds `UP.PartialThreshold` (default: 150), use `UP.PartialThreshold / RolledLoopSize`.
  - ✓ Factor 5 fits the size limit.

Example

```
for (int i = 0; i < 12; ++i) {
  foo(i); foo(i); foo(i);
  foo(i); foo(i); foo(i);
  foo(i); foo(i); foo(i);
  foo(i); foo(i); foo(i);
}
```

Trip count: **12**

Rolled loop size: **27** cost units

$$\begin{aligned}\text{Candidate unroll factor} &= \lfloor \texttt{UP.PartialThreshold}/\texttt{RolledLoopSize} \rfloor \\ &= \lfloor 150/27 \rfloor \\ &= \mathbf{5}\end{aligned}$$

Estimated unrolled loop size $= 27 \times 5 = 135$

$135 \leq 150$: Fits the size limit.

Partial Unrolling Heuristics

Trip-Count Requirements

- Trip count is a compile-time constant.
  - ✓ Trip count: **12**.

Heuristics

- If **estimated unrolled loop size** exceeds `UP.PartialThreshold` (default: 150), use `UP.PartialThreshold / RolledLoopSize`.
  - ✓ Factor 5 fits the size limit.
- Decrease `Factor` until it divides `TripCount`.
  - ✓ Candidate factor 4 divides the trip count.

Current candidate unroll factor **5** does not divide 12.

$12 = 4 \times 3$: candidate factor **4**.

Example

```
for (int i = 0; i < 12; ++i) {
  foo(i); foo(i); foo(i);
  foo(i); foo(i); foo(i);
  foo(i); foo(i); foo(i);
  foo(i); foo(i); foo(i);
}
```

Trip count: **12**

Rolled loop size: **27** cost units

Partial Unrolling Heuristics

Trip-Count Requirements

- Trip count is a compile-time constant.
  - ✓ Trip count: **12**.

Heuristics

- If **estimated unrolled loop size** exceeds `UP.PartialThreshold` (default: 150), use `UP.PartialThreshold / RolledLoopSize`.
  - ✓ Factor 5 fits the size limit.
- Decrease `Factor` until it divides `TripCount`.
  - ✓ Candidate factor 4 divides the trip count.

Example

```
for (int i = 0; i < 12; ++i) {
  foo(i); foo(i); foo(i);
  foo(i); foo(i); foo(i);
  foo(i); foo(i); foo(i);
  foo(i); foo(i); foo(i);
}
```

Trip count: **12**

Rolled loop size: **27** cost units

Conclusion

Partially unrolled by a factor of 4.

Partial Unrolling Heuristics

Trip-Count Requirements

- Trip count is a compile-time constant.
  - ✓ Trip count: **12**.

Heuristics

- If **estimated unrolled loop size** exceeds `UP.PartialThreshold` (default: 150), use `UP.PartialThreshold / RolledLoopSize`.
  - ✓ Factor 5 fits the size limit.
- Decrease `Factor` until it divides `TripCount`.
  - ✓ Candidate factor 4 divides the trip count.
- If no factor was found, start at `UP.DefaultUnrollRuntimeCount` (default: 8); halve until estimated unrolled loop size fits.
  - — Fallback not needed.

Example

```
for (int i = 0; i < 12; ++i) {
  foo(i); foo(i); foo(i);
  foo(i); foo(i); foo(i);
  foo(i); foo(i); foo(i);
  foo(i); foo(i); foo(i);
}
```

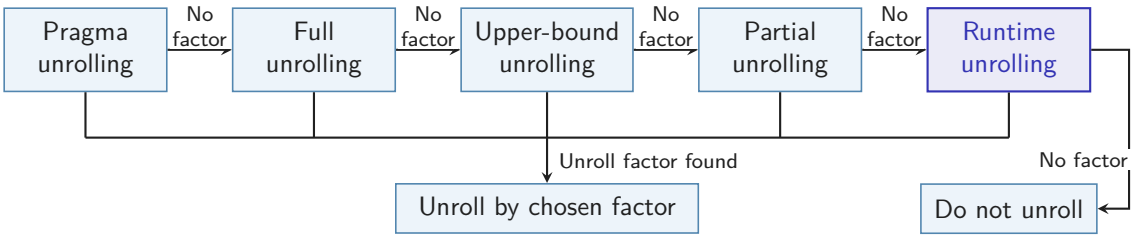
Trip count: **12**

Rolled loop size: **27** cost units

Conclusion

Partially unrolled by a factor of 4.

Unrolling Decision Order



Runtime Unrolling Heuristics

Example

```
for (uint64_t i = n; i > 2; i -= 3) {
  foo(i); foo(i); foo(i);
  foo(i); foo(i); foo(i);
  foo(i); foo(i); foo(i);
  foo(i); foo(i); foo(i);
}
```

Runtime trip count:

```
(1 + ((-3 + %n) /u 3))
```

Rolled loop size: **27** cost units

Runtime Unrolling Heuristics

Trip-Count Requirements

- Trip count is not a compile-time constant.
  - ✓ Trip count depends on `n`.

Example

```
for (uint64_t i = n; i > 2; i -= 3) {
  foo(i); foo(i); foo(i);
  foo(i); foo(i); foo(i);
  foo(i); foo(i); foo(i);
  foo(i); foo(i); foo(i);
}
```

Runtime trip count:

```
(1 + ((-3 + %n) /u 3))
```

Rolled loop size: **27** cost units

Runtime Unrolling Heuristics

Trip-Count Requirements

- Trip count is not a compile-time constant.
  - ✓ Trip count depends on `n`.

Heuristics

- Skip if known `MaxTripCount` $\leq$ `UP.MaxUpperBound` (default: 8).
  - ✓ No small proven bound.

Example

```
for (uint64_t i = n; i > 2; i -= 3) {
  foo(i); foo(i); foo(i);
  foo(i); foo(i); foo(i);
  foo(i); foo(i); foo(i);
  foo(i); foo(i); foo(i);
}
```

Runtime trip count:

```
(1 + ((-3 + %n) /u 3))
```

Rolled loop size: **27** cost units

Runtime Unrolling Heuristics

Trip-Count Requirements

- Trip count is not a compile-time constant.
  - ✓ Trip count depends on `n`.

Heuristics

- Skip if known `MaxTripCount` $\leq$ `UP.MaxUpperBound` (default: 8).
  - ✓ No small proven bound.
- Start at `UP.DefaultUnrollRuntimeCount` (default: 8).

Example

```
for (uint64_t i = n; i > 2; i -= 3) {
  foo(i); foo(i); foo(i);
  foo(i); foo(i); foo(i);
  foo(i); foo(i); foo(i);
  foo(i); foo(i); foo(i);
}
```

Runtime trip count:

```
(1 + ((-3 + %n) /u 3))
```

Rolled loop size: **27** cost units

Estimated unrolled loop size = rolled loop size $\times$ unroll factor
Factor 8: $27 \times 8 = \mathbf{216}$
$216 > 150$: Try a smaller factor.

Runtime Unrolling Heuristics

Trip-Count Requirements

- Trip count is not a compile-time constant.
  - ✓ Trip count depends on `n`.

Heuristics

- Skip if known `MaxTripCount` $\leq$ `UP.MaxUpperBound` (default: 8).
  - ✓ No small proven bound.
- Start at `UP.DefaultUnrollRuntimeCount` (default: 8).
- Halve until **estimated unrolled loop size** $\leq$ `UP.PartialThreshold` (default: 150).
  - ✓ Candidate factor 4 fits the size limit.

Example

```
for (uint64_t i = n; i > 2; i -= 3) {
  foo(i); foo(i); foo(i);
  foo(i); foo(i); foo(i);
  foo(i); foo(i); foo(i);
  foo(i); foo(i); foo(i);
}
```

Runtime trip count:

```
(1 + ((-3 + %n) /u 3))
```

Rolled loop size: **27** cost units

Estimated unrolled loop size = rolled loop size $\times$ unroll factor

Factor 8: $27 \times 8 = \mathbf{216}$

$216 > 150$: Try a smaller factor.

Factor 4: $27 \times 4 = \mathbf{108}$

$108 \leq 150$: Fits the size limit.

Runtime Unrolling Heuristics

Trip-Count Requirements

- Trip count is not a compile-time constant.
  - ✓ Trip count depends on `n`.

Heuristics

- Skip if known `MaxTripCount` $\leq$ `UP.MaxUpperBound` (default: 8).
  - ✓ No small proven bound.
- Start at `UP.DefaultUnrollRuntimeCount` (default: 8).
- Halve until **estimated unrolled loop size** $\leq$ `UP.PartialThreshold` (default: 150).
  - ✓ Candidate factor 4 fits the size limit.
- Trip-count materialization cost $\leq$ `UP.SCEVExpansionBudget` (default: 4).
  - × Materialization cost exceeds the budget.

Runtime trip-count materialization cost $>$ `UP.SCEVExpansionBudget`

$5 > 4$

Example

```
for (uint64_t i = n; i > 2; i -= 3) {
  foo(i); foo(i); foo(i);
  foo(i); foo(i); foo(i);
  foo(i); foo(i); foo(i);
  foo(i); foo(i); foo(i);
}
```

Runtime trip count:

```
(1 + ((-3 + %n) /u 3))
```

Rolled loop size: **27** cost units

Runtime Unrolling Heuristics

Trip-Count Requirements

- Trip count is not a compile-time constant.
  - ✓ Trip count depends on `n`.

Heuristics

- Skip if known `MaxTripCount` $\leq$ `UP.MaxUpperBound` (default: 8).
  - ✓ No small proven bound.
- Start at `UP.DefaultUnrollRuntimeCount` (default: 8).
- Halve until **estimated unrolled loop size** $\leq$ `UP.PartialThreshold` (default: 150).
  - ✓ Candidate factor 4 fits the size limit.
- Trip-count materialization cost $\leq$ `UP.SCEVExpansionBudget` (default: 4).
  - × Materialization cost exceeds the budget.

Example

```
for (uint64_t i = n; i > 2; i -= 3) {
  foo(i); foo(i); foo(i);
  foo(i); foo(i); foo(i);
  foo(i); foo(i); foo(i);
  foo(i); foo(i); foo(i);
}
```

Runtime trip count:

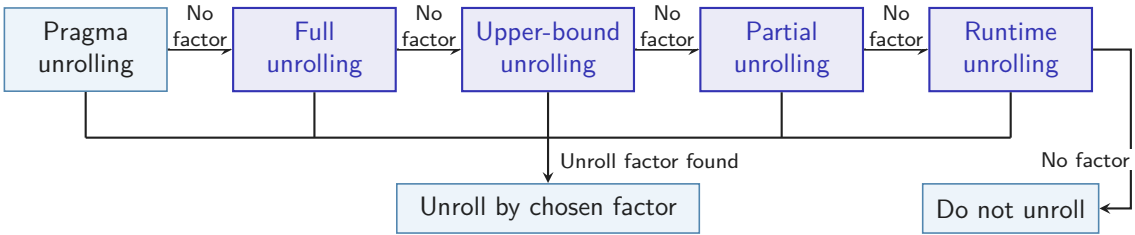
```
(1 + ((-3 + %n) /u 3))
```

Rolled loop size: **27** cost units

Conclusion

Reject runtime unrolling. Keep the original loop.

Unrolling Decision Order



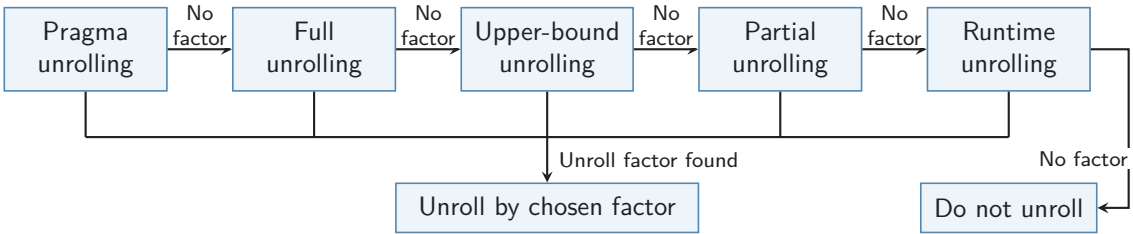
- The shared starting point is **estimated unrolled code size**.
  - Full and upper-bound unrolling use `UP.Threshold` (default: 300).
  - Partial and runtime unrolling use `UP.PartialThreshold` (default: 150).

Why Use Unrolled Code Size?

Larger unrolled loops take more work to compile and put more pressure on the instruction cache.

- **Compile time**: limit IR growth to keep later passes fast.
  - Cold loops offer little speedup, but extra copies still cost compile time.
- **Cleanup**: favor small loop bodies, where overhead often matters more and simplification can justify code growth.
- **Instruction cache**: limit code growth to avoid i-cache thrashing.

Unrolling Decision Order



- Honor the programmer's unrolling requests when possible.
- Otherwise, use automatic unrolling heuristics.
  - Their key heuristic is **estimated unrolled code size**.

Table of Contents

Background

Unrolling Different Kinds of Loops

Deciding to Unroll

Pass Interactions at -O3

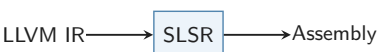
Notes for the Road

Pass Interactions at -O3

`opt -O3`



`llc -O3` (NVPTX)



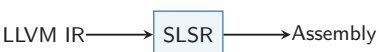
Other passes are omitted for brevity; shown passes retain their order.

Pass Interactions at -O3

`opt -O3`



`llc -O3` (NVPTX)



Other passes are omitted for brevity; shown passes retain their order.

Scalar Replacement of Aggregates (SROA): Stack Accesses

- Unrolling **enables other optimizations**, including SROA.
- SROA tries to eliminate stack allocations and their loads and stores.

C Source

```
unsigned a[2];
for (unsigned i = 0; i < 2; ++i)
  a[i] = x + i;
return a[0] + a[1];
```

Before Unrolling

```
entry:
  %a = alloca [2 x i32], align 4
  br label %loop
loop:
  %i = phi i32 [ 0, %entry ], [ %next, %loop ]
  %p = getelementptr [2 x i32], ptr %a, i32 0, i32 %i
  %v = add i32 %x, %i
  store i32 %v, ptr %p, align 4
  %next = add nuw nsw i32 %i, 1
  %more = icmp ult i32 %next, 2
  br i1 %more, label %loop, label %exit
exit:
  %p1 = getelementptr [2 x i32], ptr %a, i32 0, i32 1
  %v0 = load i32, ptr %a, align 4
  %v1 = load i32, ptr %p1, align 4
  %sum = add i32 %v0, %v1
  ret i32 %sum
```

SROA: Variable Indices

- SROA cannot promote this array while the GEP index `%i` is variable.
- However, SROA can eliminate stack allocations accessed through constant indices.

C Source

```
unsigned a[2];
for (unsigned i = 0; i < 2; ++i)
  a[i] = x + i;
return a[0] + a[1];
```

Before Unrolling

```
entry:
  %a = alloca [2 x i32], align 4
  br label %loop
loop:
  %i = phi i32 [ 0, %entry ], [ %next, %loop ]
  %p = getelementptr [2 x i32], ptr %a, i32 0, i32 %i
  %v = add i32 %x, %i
  store i32 %v, ptr %p, align 4
  %next = add nuw nsw i32 %i, 1
  %more = icmp ult i32 %next, 2
  br i1 %more, label %loop, label %exit
exit:
  %p1 = getelementptr [2 x i32], ptr %a, i32 0, i32 1
  %v0 = load i32, ptr %a, align 4
  %v1 = load i32, ptr %p1, align 4
  %sum = add i32 %v0, %v1
  ret i32 %sum
```

SROA: Constant Indices After Unrolling

- Full unrolling makes the array indices constant.

Before Unrolling

```
entry:
  %a = alloca [2 x i32], align 4
  br label %loop
loop:
  %i = phi i32 [ 0, %entry ], [ %next, %loop ]
  %p = getelementptr [2 x i32], ptr %a, i32 0, i32 %i
  %v = add i32 %x, %i
  store i32 %v, ptr %p, align 4
  %next = add nuw nsw i32 %i, 1
  %more = icmp ult i32 %next, 2
  br i1 %more, label %loop, label %exit
exit:
  %p1 = getelementptr [2 x i32], ptr %a, i32 0, i32 1
  %v0 = load i32, ptr %a, align 4
  %v1 = load i32, ptr %p1, align 4
  %sum = add i32 %v0, %v1
  ret i32 %sum
```

After Full Unrolling

```
%a = alloca [2 x i32], align 4
store i32 %x, ptr %a, align 4
%p.1 = getelementptr [2 x i32], ptr %a, i32 0, i32 1
%v.1 = add i32 %x, 1
store i32 %v.1, ptr %p.1, align 4
%p1 = getelementptr [2 x i32], ptr %a, i32 0, i32 1
%v0 = load i32, ptr %a, align 4
%v1 = load i32, ptr %p1, align 4
%sum = add i32 %v0, %v1
ret i32 %sum
```

SROA: Removing Stack Accesses

- Full unrolling makes the array indices constant.
- SROA removes the allocation, stores, and loads. The values stay in SSA.

Before Unrolling

```
entry:
  %a = alloca [2 x i32], align 4
  br label %loop
loop:
  %i = phi i32 [ 0, %entry ], [ %next, %loop ]
  %p = getelementptr [2 x i32], ptr %a, i32 0, i32 %i
  %v = add i32 %x, %i
  store i32 %v, ptr %p, align 4
  %next = add nuw nsw i32 %i, 1
  %more = icmp ult i32 %next, 2
  br i1 %more, label %loop, label %exit
exit:
  %p1 = getelementptr [2 x i32], ptr %a, i32 0, i32 1
  %v0 = load i32, ptr %a, align 4
  %v1 = load i32, ptr %p1, align 4
  %sum = add i32 %v0, %v1
  ret i32 %sum
```

After Full Unrolling

```
%a = alloca [2 x i32], align 4
store i32 %x, ptr %a, align 4
%p.1 = getelementptr [2 x i32], ptr %a, i32 0, i32 1
%v.1 = add i32 %x, 1
store i32 %v.1, ptr %p.1, align 4
%p1 = getelementptr [2 x i32], ptr %a, i32 0, i32 1
%v0 = load i32, ptr %a, align 4
%v1 = load i32, ptr %p1, align 4
%sum = add i32 %v0, %v1
ret i32 %sum
```

After SROA

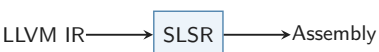
```
%v.1 = add i32 %x, 1
%sum = add i32 %x, %v.1
ret i32 %sum
```

Pass Interactions at -O3

`opt -O3`



`llc -O3` (NVPTX)



Other passes are omitted for brevity; shown passes retain their order.

AlignmentFromAssumptions: The Rolled Loop

- `llvm.assume` says %p is **16-byte aligned**.
- One store executes four times. Its alignment must hold for every `p[i]`.

C Source

```
p = __builtin_assume_aligned(p, 16);

#pragma clang loop unroll(full)
for (int i = 0; i < 4; ++i)
  p[i] = 7;
```

Rolled Loop

```
entry:
  call void @llvm.assume(i1 true) ["align"(ptr %p, i64 16)]
  br label %loop

loop:
  %i = phi i64 [ 0, %entry ], [ %next, %loop ]
  %addr = getelementptr i32, ptr %p, i64 %i
  store i32 7, ptr %addr, align 4
  %next = add nuw nsw i64 %i, 1
  %continue = icmp slt i64 %next, 4
  br i1 %continue, label %loop, label %exit

exit:
  ret void
```

AlignmentFromAssumptions: The Rolled Loop

- `llvm.assume` says %p is **16-byte aligned**.
- One store executes four times. Its alignment must hold for every `p[i]`.

C Source

```
p = __builtin_assume_aligned(p, 16);

#pragma clang loop unroll(full)
for (int i = 0; i < 4; ++i)
  p[i] = 7;
```

Rolled Loop

```
entry:
  call void @llvm.assume(i1 true) ["align"(ptr %p, i64 16)]
  br label %loop

loop:
  %i = phi i64 [ 0, %entry ], [ %next, %loop ]
  %addr = getelementptr i32, ptr %p, i64 %i
  store i32 7, ptr %addr, align 4
  %next = add nuw nsw i64 %i, 1
  %continue = icmp slt i64 %next, 4
  br i1 %continue, label %loop, label %exit

exit:
  ret void
```

AlignmentFromAssumptions: The Rolled Loop

- `llvm.assume` says %p is **16-byte aligned**.
- One store executes four times. Its alignment must hold for every `p[i]`.

C Source

```
p = __builtin_assume_aligned(p, 16);

#pragma clang loop unroll(full)
for (int i = 0; i < 4; ++i)
  p[i] = 7;
```

- `p[0]`: **16-byte aligned**
- `p[1]`: **4-byte aligned**
- `p[2]`: **8-byte aligned**
- `p[3]`: **4-byte aligned**

Rolled Loop

```
entry:
  call void @llvm.assume(i1 true) ["align"(ptr %p, i64 16)]
  br label %loop

loop:
  %i = phi i64 [ 0, %entry ], [ %next, %loop ]
  %addr = getelementptr i32, ptr %p, i64 %i
  store i32 7, ptr %addr, align 4
  %next = add nuw nsw i64 %i, 1
  %continue = icmp slt i64 %next, 4
  br i1 %continue, label %loop, label %exit

exit:
  ret void
```

AlignmentFromAssumptions: Constant Offsets

- Full unrolling gives each store a fixed offset from %p.
- These i32 accesses are **0, 4, 8, and 12 bytes** from the aligned base.

Before Full Unrolling

```
entry:
  call void @llvm.assume(i1 true) ["align"(ptr %p, i64 16)]
  br label %loop

loop:
  %i = phi i64 [ 0, %entry ], [ %next, %loop ]
  %addr = getelementptr i32, ptr %p, i64 %i
  store i32 7, ptr %addr, align 4
  %next = add nuw nsw i64 %i, 1
  %continue = icmp slt i64 %next, 4
  br i1 %continue, label %loop, label %exit

exit:
  ret void
```

Unrolled IR

```
entry:
  call void @llvm.assume(i1 true) ["align"(ptr %p, i64 16)]
  store i32 7, ptr %p, align 4
  %p1 = getelementptr i32, ptr %p, i64 1
  store i32 7, ptr %p1, align 4
  %p2 = getelementptr i32, ptr %p, i64 2
  store i32 7, ptr %p2, align 4
  %p3 = getelementptr i32, ptr %p, i64 3
  store i32 7, ptr %p3, align 4
  ret void
```

AlignmentFromAssumptions: Stronger Alignment

- The pass combines the assumed base alignment with each store's offset.
- The guarantees become **16, 4, 8, and 4 bytes**. The accesses stay the same.

Unrolled IR

```
entry:
  call void @llvm.assume(i1 true) ["align"(ptr %p, i64 16)]
  store i32 7, ptr %p, align 4
  %p1 = getelementptr i32, ptr %p, i64 1
  store i32 7, ptr %p1, align 4
  %p2 = getelementptr i32, ptr %p, i64 2
  store i32 7, ptr %p2, align 4
  %p3 = getelementptr i32, ptr %p, i64 3
  store i32 7, ptr %p3, align 4
  ret void
```

After AlignmentFromAssumptions

```
entry:
  call void @llvm.assume(i1 true) ["align"(ptr %p, i64 16)]
  store i32 7, ptr %p, align 16
  %p1 = getelementptr i32, ptr %p, i64 1
  store i32 7, ptr %p1, align 4
  %p2 = getelementptr i32, ptr %p, i64 2
  store i32 7, ptr %p2, align 8
  %p3 = getelementptr i32, ptr %p, i64 3
  store i32 7, ptr %p3, align 4
  ret void
```

Pass Interactions at -O3

`opt -O3`



`llc -O3` (NVPTX)



Other passes are omitted for brevity; shown passes retain their order.

Straight-Line Strength Reduction (SLSR): Repeated Arithmetic

- Unrolled code often repeats related arithmetic and address calculations.
- SLSR **optimizes this unrolled code** by reusing an earlier result.

C Source

```
for (uint64_t i = 0; i < 2; ++i)
  a[(start + i) * stride] = 7;
```

Before Unrolling

```
entry:
  br label %loop
loop:
  %i = phi i64 [ 0, %entry ], [ %next, %loop ]
  %index = add i64 %start, %i
  %offset = mul i64 %index, %stride
  %p = getelementptr i8, ptr %a, i64 %offset
  store i8 7, ptr %p, align 1
  %next = add nuw nsw i64 %i, 1
  %more = icmp ult i64 %next, 2
  br i1 %more, label %loop, label %exit
exit:
  ret void
```

SLSR: Repeated Offset Calculations

- Iteration 0: $(\texttt{start} + 0) \times \texttt{stride}$
- Iteration 1: $(\texttt{start} + 1) \times \texttt{stride}$

Before Unrolling

```
entry:
  br label %loop
loop:
  %i = phi i64 [ 0, %entry ], [ %next, %loop ]
  %index = add i64 %start, %i
  %offset = mul i64 %index, %stride
  %p = getelementptr i8, ptr %a, i64 %offset
  store i8 7, ptr %p, align 1
  %next = add nuw nsw i64 %i, 1
  %more = icmp ult i64 %next, 2
  br i1 %more, label %loop, label %exit
exit:
  ret void
```

SLSR: Repeated Offset Calculations

- Iteration 0: $(\texttt{start} + 0) \times \texttt{stride}$
- Iteration 1: $(\texttt{start} + 1) \times \texttt{stride}$

Before Unrolling

```
entry:
  br label %loop
loop:
  %i = phi i64 [ 0, %entry ], [ %next, %loop ]
  %index = add i64 %start, %i
  %offset = mul i64 %index, %stride
  %p = getelementptr i8, ptr %a, i64 %offset
  store i8 7, ptr %p, align 1
  %next = add nuw nsw i64 %i, 1
  %more = icmp ult i64 %next, 2
  br i1 %more, label %loop, label %exit
exit:
  ret void
```

- Iteration 0: $\texttt{start} \times \texttt{stride}$
- Iteration 1: $(\texttt{start} + 1) \times \texttt{stride}$

Unrolled Body Before SLSR

```
%offset = mul i64 %start, %stride
%p = getelementptr i8, ptr %a, i64 %offset
store i8 7, ptr %p, align 1
%index.1 = add i64 %start, 1
%offset.1 = mul i64 %index.1, %stride
%p.1 = getelementptr i8, ptr %a, i64 %offset.1
store i8 7, ptr %p.1, align 1
ret void
```

SLSR: Reusing the First Offset

- Iteration 0: $\texttt{start} \times \texttt{stride}$
- Iteration 1: $(\texttt{start} + 1) \times \texttt{stride}$

Unrolled Body Before SLSR

```
%offset = mul i64 %start, %stride
%p = getelementptr i8, ptr %a, i64 %offset
store i8 7, ptr %p, align 1
%index.1 = add i64 %start, 1
%offset.1 = mul i64 %index.1, %stride
%p.1 = getelementptr i8, ptr %a, i64 %offset.1
store i8 7, ptr %p.1, align 1
```

- Iteration 0: $\texttt{start} \times \texttt{stride}$
- Iteration 1: $(\texttt{start} \times \texttt{stride}) + \texttt{stride}$

Unrolled Body After SLSR

```
%offset = mul i64 %start, %stride
%p = getelementptr i8, ptr %a, i64 %offset
store i8 7, ptr %p, align 1
%offset.1 = add i64 %offset, %stride
%p.1 = getelementptr i8, ptr %a, i64 %offset.1
store i8 7, ptr %p.1, align 1
```

$$(\texttt{start} + 1) \times \texttt{stride} \overset{\text{distribute}}{=} (\texttt{start} \times \texttt{stride}) + \texttt{stride}$$

- Iteration 1 **reuses the offset** computed by iteration 0.
- This eliminates **one multiply** in this example.

Table of Contents

Background

Unrolling Different Kinds of Loops

Deciding to Unroll

Pass Interactions at -O3

Notes for the Road

More to Explore

- **Loop Shapes and Preparation**
  Nested or irreducible loop shapes, rotation, and induction-variable simplification.
- **Unrolling Decisions**
  The full size calculation, `analyzeLoopUnrollCost()`, profiling, and convergence constraints.
- **Rewriting the Loop**
  Cloning and value remapping, runtime setup, epilogue vs prologue remainders, alternative forms of remainder unrolling, handling `continue` statements, and multiple-exit or overflow cases.
- **Related Loop Transformations**
  Peeling, unswitching, unroll-and-jam, rerolling, and partial accumulators.
- **Scalar Pass Interactions**
  Inlining, LICM, GVN, InstCombine, DSE, and SimplifyCFG.
- **Vectorization and Addressing**
  LoopVectorizer, SLP, load/store vectorization, and constant-offset separation.

Notes for the Road

Where to Start

Your Question	Start Here
Why this form and factor?	`Scalar/LoopUnrollPass.cpp`
How did the loop's IR change?	`Utils/LoopUnroll.cpp`
Why these guards and remainder paths?	`Utils/LoopUnrollRuntime.cpp`

- **Thank you** to Aidan Goldfarb, Alex MacLean, and Nadharm Dhiantravan for help preparing the talk.

- Slides: `justinfargnoli.github.io/slides/unroll.pdf`

Help Us Shape the Future

US LLVM Developers' Meeting 2026

Scan the QR code to connect with us.
NVIDIA is the engine of the world's AI infrastructure. We are the world leader in accelerated computing.

Check out our open career opportunities here:
https://www.nvidia.com/en-us/about-nvidia/careers/



Backup: Partial Unrolling with Continue

Source

```
for (int i = 0; i < 100; ++i) {
  if (skip(i))
    continue;
  foo(i);
}
```

- `continue` skips only the current iteration's call.
- The first copy advances to the second copy; the second copy reaches the loop's backedge test.

After unrolling by a factor of 2

```
body:
  %i = phi i32 [ 0, %entry ], [ %inc.1, %latch.1 ]
  %skip = call i1 @skip(i32 noundef %i)
  br i1 %skip, label %latch, label %call
call:
  call void @foo(i32 noundef %i)
  br label %latch
latch:
  %inc = add nuw nsw i32 %i, 1
  %skip.1 = call i1 @skip(i32 noundef %inc)
  br i1 %skip.1, label %latch.1, label %call.1
call.1:
  call void @foo(i32 noundef %inc)
  br label %latch.1
latch.1:
  %inc.1 = add nuw nsw i32 %i, 2
  %cmp.1 = icmp slt i32 %inc.1, 100
  br i1 %cmp.1, label %body, label %exit
exit:
  ret void
```

Backup: Partial Unrolling with Break

Source

```
for (int i = 0; i < 100; ++i) {
  if (stop(i))
    break;
  foo(i);
}
```

- Keep a `break` check in each copy.
- A true check exits the whole loop; later copies do not execute.

After unrolling by a factor of 2

```
body:
  %i = phi i32 [ 0, %entry ], [ %inc.1, %latch.1 ]
  %stopped = call zeroext i1 @stop(i32 noundef %i)
  br i1 %stopped, label %exit, label %latch
latch:
  call void @foo(i32 noundef %i)
  %inc = add nuw nsw i32 %i, 1
  %stopped.1 = call zeroext i1 @stop(i32 noundef %inc)
  br i1 %stopped.1, label %exit, label %latch.1
latch.1:
  call void @foo(i32 noundef %inc)
  %inc.1 = add nuw nsw i32 %i, 2
  %cmp.1 = icmp slt i32 %inc.1, 100
  br i1 %cmp.1, label %body, label %exit
exit:
  ret void
```

Backup: An Uneven Trip Count

Source

```
for (int i = 0; i < 5; ++i)
  foo(i);
```

- The trip count, 5, is not divisible by the unroll factor, 2.
- Keep an exit after the first copy: the final group stops after `foo(4)`.
- This output needs no separate remainder loop.

After unrolling by a factor of 2

```
body:
  %i = phi i32 [ 0, %entry ], [ %inc.1, %body.1 ]
  call void @foo(i32 noundef %i)
  %inc = add nuw nsw i32 %i, 1
  %cmp = icmp slt i32 %inc, 5
  br i1 %cmp, label %body.1, label %exit
body.1:
  call void @foo(i32 noundef %inc)
  %inc.1 = add nuw nsw i32 %i, 2
  br label %body
exit:
  ret void
```